

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA

FACULTAD DE INGENIERIA DE SISTEMAS

CURSO: BASE DE DATOS .

ING: ORNELIS

TAREA: AUTOAUDIT

MANUAL DE USUARIO

ESTUDIANTE: BREYNER ORLANDO LARIOS SERECH

CARNET: 1990-23-1411

AutoAudit – Sistema de auditoría automatizado

Manual técnico

Repositorio: https://github.com/blarioss-prog/autoaudit

1. Descripción general

AutoAudit es un sistema de auditoría configurable para PostgreSQL. Un administrador elige qué tablas, columnas y eventos (INSERT, UPDATE, DELETE) se auditan. A partir de esa configuración, el sistema genera dinámicamente triggers que registran cada cambio en una bitácora central, con el valor antiguo, el valor nuevo, el usuario, la estación de trabajo (IP) y la fecha y hora.

Todo el sistema vive en un esquema propio llamado `aud`, de modo que no interfiere con las tablas del negocio.

1.1 Objetivos

- Capturar cambios en tablas configurables mediante triggers generados automáticamente.
- Permitir la consulta histórica de modificaciones por usuario o por columna.
- Conservar los datos originales en la bitácora aunque el registro fuente sea eliminado.

1.2 Entorno de desarrollo y pruebas

Componente	Detalle
Sistema operativo	Ubuntu Desktop (máquina virtual en VirtualBox)
Base de datos	PostgreSQL 16.15
Cliente SQL	DBeaver 26.2.4
Lenguaje de programación	SQL y PL/pgSQL (el equivalente de PL/SQL en PostgreSQL)
Control de versiones	Git / GitHub

Nota: el enunciado indica PostgreSQL 18 y Ubuntu 26.04 LTS. El sistema usa solo características estándar de PL/pgSQL, por lo que debe funcionar igual en PostgreSQL 18; la verificación en esa versión queda pendiente de confirmar.

2. Arquitectura

El sistema se compone de cuatro módulos:

Módulo	Objetos principales	Función
Configuración	`aud.tabla`, `aud.columna`, `aud.configuracion`, `aud.configurar()`	Guarda qué se audita y valida que tablas y columnas existan

Módulo	Objetos principales	Función
Generación de triggers	`aud.generar_trigger()`	Lee la configuración y crea la función y el trigger de una tabla
Bitácora	`aud.bitacora`, `aud.usuario`, `aud.estacion`	Almacena cada cambio con su contexto
Consulta y reporte	`aud.historial_usuario()`, `aud.historial_columna()`	Consultas parametrizadas del historial

2.1 Flujo de funcionamiento

1. El administrador llama a `aud.configurar()` indicando tabla, columnas y eventos.
2. La función valida contra el diccionario de datos (`information_schema`) y guarda la configuración.
3. El administrador llama a `aud.generar_trigger()` para esa tabla.
4. Se crea un trigger `AFTER` que, en cada INSERT, UPDATE o DELETE, ejecuta la función generada.
5. La función escribe una fila en `aud.bitacora` por cada columna auditada afectada.
6. El historial se consulta con las funciones de reporte.

3. Modelo entidad-relación

[Inserta aquí la imagen `diagrama_er.png`, generada en DBeaver: selecciona las 7 tablas del esquema `aud`, clic derecho, Ver diagrama, y guarda como imagen.]

3.1 Relaciones entre entidades

Relación	Descripción
`aud.tabla` 1 → N `aud.columna`	Una tabla auditada tiene varias columnas auditadas
`aud.columna` 1 → N `aud.configuracion`	Cada columna tiene una configuración por evento
`aud.evento` 1 → N `aud.configuracion`	El catálogo de eventos (INSERT, UPDATE, DELETE)
`aud.configuracion` 1 → N `aud.bitacora`	Cada registro de bitácora referencia la configuración usada
`aud.usuario` 1 → N `aud.bitacora`	Usuario responsable del cambio
`aud.estacion` 1 → N `aud.bitacora`	Estación de trabajo desde donde se hizo el cambio

La bitácora **no tiene llave foránea hacia las filas auditadas**. Es una decisión de diseño intencional: así los datos originales se conservan aunque el registro fuente se elimine.

4. Diccionario de datos

4.1 aud.evento

Catálogo de eventos auditables. Se carga con tres filas: 1 INSERT, 2 UPDATE, 3 DELETE.

Columna	Tipo	Nulo	Descripción
id	SMALLINT (PK)	No	Identificador del evento
codigo	TEXT (único)	No	Nombre del evento: INSERT, UPDATE o DELETE

4.2 aud.tabla

Tablas que han sido configuradas para auditoría.

Columna	Tipo	Nulo	Descripción
id	SERIAL (PK)	No	Identificador
esquema	TEXT	No	Esquema de la tabla auditada
nombre	TEXT	No	Nombre de la tabla auditada
activa	BOOLEAN	No	Indica si la auditoría de la tabla está activa (por defecto TRUE)

Restricción: `UNIQUE (esquema, nombre)`.

4.3 aud.columna

Columnas auditadas de cada tabla.

Columna	Tipo	Nulo	Descripción
id	SERIAL (PK)	No	Identificador
tabla_id	INT (FK → aud.tabla)	No	Tabla a la que pertenece
nombre	TEXT	No	Nombre de la columna

Restricción: `UNIQUE (tabla_id, nombre)`.

4.4 aud.configuracion

Qué evento se audita en qué columna.

Columna	Tipo	Nulo	Descripción
id	SERIAL (PK)	No	Identificador
columna_id	INT (FK → aud.columna)	No	Columna auditada
evento_id	SMALLINT (FK → aud.evento)	No	Evento auditado
activa	BOOLEAN	No	Activa o desactiva este evento (por defecto TRUE)

Restricción: `UNIQUE (columna_id, evento_id)`.

4.5 aud.usuario

Columna	Tipo	Nulo	Descripción
id	SERIAL (PK)	No	Identificador
nombre	TEXT (único)	No	Usuario de base de datos que hizo el cambio

4.6 aud.estacion

Columna	Tipo	Nulo	Descripción
id	SERIAL (PK)	No	Identificador
ip	TEXT (único)	No	IP del cliente; `local` si la conexión es por socket

4.7 aud.bitacora

Tabla central de registro de cambios. Cada fila corresponde a **una columna modificada** en una operación.

Columna	Tipo	Nulo	Descripción
id	BIGSERIAL (PK)	No	Identificador
configuracion_id	INT (FK → aud.configuracion)	No	Configuración que originó el registro
usuario_id	INT (FK → aud.usuario)	No	Usuario responsable
estacion_id	INT (FK → aud.estacion)	No	Estación de trabajo
fecha_hora	TIMESTAMPTZ	No	Momento del cambio (por defecto `now()`)
pk_registro	TEXT	Sí	Llave primaria del registro afectado
valor_antiguo	TEXT	Sí	Valor antes del cambio (UPDATE y DELETE)
valor_nuevo	TEXT	Sí	Valor después del cambio (INSERT y UPDATE)

Índices: `ix_bitacora_usuario (usuario_id, fecha_hora)` e `ix_bitacora_config (configuracion_id, fecha_hora)`, para acelerar las consultas por usuario y por columna.

5. Procedimiento de instalación y despliegue de triggers

5.1 Requisitos

- Ubuntu con PostgreSQL instalado y un usuario con privilegios administrativos.
- Un cliente SQL (DBeaver o `psql`).

5.2 Instalar PostgreSQL

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib git
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'tu_clave';"
```

5.3 Crear la base de datos

```
CREATE DATABASE autoaudit;
```

5.4 Ejecutar los scripts en orden

Conectado a la base `autoaudit`, ejecuta los archivos del repositorio en este orden:

Orden	Archivo	Contenido
1	`01_tablas.sql`	Esquema `aud` y las 7 tablas del sistema
2	`02_diccionario.sql`	Consultas al diccionario de datos
3	`03_configuracion.sql`	Función `aud.configurar` (y tabla de práctica `cliente`)
4	`04_generador_triggers.sql`	Generador dinámico de triggers y funciones auxiliares
5	`05_consultas.sql`	Funciones de historial por usuario y por columna
6	`06_pruebas.sql`	Casos de prueba

Desde la terminal también puede hacerse con `psql`:

```
sudo -u postgres psql -d autoaudit -f 01_tablas.sql
```

Verificación: `\dt aud.*` debe listar las 7 tablas, y `SELECT * FROM aud.evento;` debe devolver 3 filas.

5.5 Despliegue de triggers

Una vez instalado el sistema, los triggers se despliegan tabla por tabla con este procedimiento:

1. **Configurar** la tabla, sus columnas y sus eventos con `aud.configurar()` (sección 6.2).
2. **Generar** el trigger con `aud.generar_trigger()` (sección 6.3).
3. **Verificar** que el trigger quedó instalado y habilitado:

```
SELECT tgname AS trigger, tgrelid::regclass AS tabla, tgenabled AS habilitado
FROM pg_trigger
WHERE tgname LIKE 'trg_aud_%';
```

4. **Probar** con una operación real sobre la tabla y revisar que aparezca en `aud.bitacora` (sección 10.1).
5. **Regenerar** el trigger con `aud.generar_trigger()` cada vez que cambie la configuración de la tabla.
6. **Retirar** la auditoría de una tabla desactivando su configuración (sección 6.4) y ejecutando de nuevo `aud.generar_trigger()`, que elimina el trigger si no queda ningún evento activo.

6. Manual de configuración y activación de auditoría

6.1 Consultar qué tablas y columnas existen

La selección se basa en el diccionario de datos:

```
SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_type = 'BASE TABLE'
  AND table_schema NOT IN ('pg_catalog', 'information_schema', 'aud');

SELECT column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public' AND table_name = 'cliente'
ORDER BY ordinal_position;
```

6.2 Configurar la auditoría de una tabla

```
SELECT aud.configurar('public', 'cliente',
       ARRAY['nombre','telefono'],
       ARRAY['INSERT','UPDATE','DELETE']);
```

Parámetros: esquema, tabla, arreglo de columnas y arreglo de eventos. La función valida que la tabla, las columnas y los eventos existan; si no, lanza un error y no guarda nada. Por ejemplo, pedir la columna `correo` en `cliente` produce: `La columna correo no existe en public.cliente`.

Puede auditarse solo una parte de las columnas o de los eventos, por ejemplo solo UPDATE de `telefono`.

6.3 Generar el trigger

```
SELECT aud.generar_trigger('public', 'cliente');
```

Crea la función `aud.tg_public_cliente()` y el trigger `trg_aud_cliente` de tipo `AFTER ... FOR EACH ROW` sobre la tabla. Responde con los eventos cubiertos, por ejemplo: `Trigger trg_aud_cliente creado para: DELETE OR INSERT OR UPDATE`.

6.4 Modificar o desactivar la auditoría

Desactivar un evento de una columna (por ejemplo, dejar de auditar DELETE en `telefono`):

```
UPDATE aud.configuracion SET activa = FALSE
WHERE columna_id = (SELECT c.id FROM aud.columna c
                    JOIN aud.tabla t ON t.id = c.tabla_id
                    WHERE t.nombre = 'cliente' AND c.nombre = 'telefono')
  AND evento_id = (SELECT id FROM aud.evento WHERE codigo = 'DELETE');
```

Desactivar toda la auditoría de una tabla:

```
UPDATE aud.tabla SET activa = FALSE WHERE esquema = 'public' AND nombre = 'cliente';
```

Después de cualquier cambio de configuración, vuelve a ejecutar `aud.generar_trigger()` para esa tabla. Esto es necesario porque la lista de eventos del trigger se fija al generarlo. Si ya no queda ningún evento activo, `generar_trigger` elimina el trigger.

No se recomienda borrar filas de `aud.configuracion` que ya tengan registros en la bitácora, porque la llave foránea lo impide: se desactivan, no se eliminan.

7. Funcionamiento del generador de triggers

`aud.generar_trigger(esquema, tabla)` realiza estos pasos:

1. Valida que la tabla exista en el diccionario de datos.
2. Obtiene los eventos activos configurados para la tabla.
3. Detecta las columnas de la llave primaria con `pg_index`.
4. Crea con `EXECUTE format(...)` una función `aud.tg_<esquema>_<tabla>()` propia de esa tabla.
5. Elimina y recrea el trigger `trg_aud_<tabla>` con los eventos activos.

La función generada convierte `OLD` y `NEW` a JSON, y para cada columna configurada inserta una fila en `aud.bitacora`. En los UPDATE solo se registran las columnas cuyo valor realmente cambió. Cada trigger maneja exclusivamente su tabla origen, como exige el enunciado.

Funciones auxiliares: `aud.id_usuario()` obtiene o crea el usuario de sesión (`session_user`), y `aud.id_estacion()` obtiene o crea la estación a partir de `inet_client_addr()`.

8. Consultas de historial

8.1 Por usuario

```
SELECT * FROM aud.historial_usuario('postgres');
SELECT * FROM aud.historial_usuario('postgres', '2026-11-01', '2026-11-30');
```

8.2 Por columna

```
SELECT * FROM aud.historial_columna('public', 'cliente', 'telefono');
```

Ambas funciones devuelven el resultado ordenado cronológicamente, con fecha, usuario, IP, tabla, columna, evento, llave del registro, valor antiguo y valor nuevo. Los dos parámetros de fecha son opcionales.

8.3 Exportación de reportes

En DBeaver, sobre cualquier resultado: botón **Exportar datos** y elegir CSV, Excel o JSON.

9. Seguridad

- **Configuración restringida:** solo un usuario administrador debe ejecutar `aud.configurar` y `aud.generar_trigger`. Se recomienda otorgar `EXECUTE` sobre estas funciones únicamente a un rol administrativo.
- **Bitácora protegida contra escritura directa:** se revocan `INSERT`, `UPDATE` y `DELETE` sobre `aud.bitacora` a `PUBLIC`. Solo las funciones de trigger, definidas con `SECURITY DEFINER`, escriben en ella. Los superusuarios de PostgreSQL conservan acceso por naturaleza del motor.
- **Privacidad de la IP:** la IP del cliente se almacena en `aud.estacion`. Su uso debe cumplir las políticas de privacidad aplicables; si fuera necesario, puede anonimizarse o restringirse el acceso de lectura a esa tabla.
- **Conservación de datos:** la bitácora no depende de las filas auditadas, por lo que los valores de un registro eliminado se conservan.

10. Pruebas de validación

10.1 Casos funcionales

Caso	Acción	Resultado esperado
Insert	`INSERT INTO cliente (nombre, telefono) VALUES ('Ana','5555-1234')`	2 filas en bitácora (nombre y teléfono), con valor nuevo
Update	`UPDATE cliente SET telefono = '4444-0000' WHERE nombre = 'Ana'`	1 fila (solo teléfono), valor antiguo `5555-1234` y nuevo `4444-0000`
Delete	`DELETE FROM cliente WHERE nombre = 'Ana'`	2 filas con los valores originales conservados, aunque el cliente ya no exista
Validación	`aud.configurar` con columna `correo`	Error: la columna no existe
Historial por usuario	`aud.historial_usuario('postgres')`	Los 5 cambios anteriores, en orden cronológico
Historial por columna	`aud.historial_columna('public','cliente','telefono')`	Los 3 cambios del teléfono

10.2 Pruebas de rendimiento

Se compara la inserción masiva en una tabla sin auditoría (`cliente_sin_aud`) y otra con auditoría (`cliente`), con volumen creciente, mediante `aud.prueba_rendimiento(n)`.

Filas insertadas	Sin auditoría	Con auditoría
1,000	[completar]	[completar]
10,000	[completar]	[completar]
50,000	[completar]	[completar]

[Completa la tabla con los tiempos que obtuviste y agrega un párrafo de análisis: la auditoría añade un costo porque cada fila genera dos registros en la bitácora; indica cuánto más lento fue y si el costo creció de forma proporcional al volumen.]

11. Limitaciones conocidas

- La auditoría es por fila (`FOR EACH ROW`); en cargas masivas el costo es proporcional al número de filas.
- Los valores se guardan como texto, por lo que se pierde el tipo original del dato.
- Si se actualiza la llave primaria de un registro, `pk_registro` guarda el valor nuevo.
- Tras cambiar la configuración hay que regenerar el trigger de la tabla.

- No se desarrolló una interfaz gráfica; la configuración y la consulta se hacen mediante funciones SQL.